Yenepoya Institute of Arts, Science, Commerce, and Management

Project – Low Level Design

on

Cloud Forensics Automation for Manufacturing Breaches

Student Name: Rajih Ali PP

Industry Mentor: Mr. Shashank Korram **Guided by: Ms. Prathiksha Mudipu**

TABLE OF CONTENTS

Section No.	Title	Page No.
1	Introduction	3
1.1	Purpose of Document	3
1.2	Scope of the System	3
1.3	Definitions & Abbreviations	4
2	System Overview	5
2.1	System Description	5
2.2	Architecture Overview	5
2.3	Technology Stack	5
3	Module Design	6
4	Class Design & Object Interaction	7
4.1	Class Diagram	7
4.2	Class Descriptions	8
4.3	Object Initialization Flow	9
5	Sequence Diagrams	9-10
6	API Specifications	11
7	Data Design	13
8	Security Considerations	14
9	Error Handling & Edge Cases	15
10	Logging System	16
11	Deployment Considerations	17
12	Testing Strategy	18
13	Future Enhancements	19
14	Conclusion	20

1. Introduction

1.1 Purpose of Document

This document provides a detailed Low-Level Design (LLD) for the Cloud Forensics Automation for Manufacturing Breaches project. It describes the internal architecture, module-level design, class interactions, sequence flows, and component responsibilities required to implement the forensic file evidence collection system.

The purpose of this document is to:

- Define detailed system components, functions, and their responsibilities
- Describe file scanning, SHA-256/MD5 hash computation, and timestamp anomaly detection flows
- Provide technical design for developers and evaluators
- Document API specifications and Streamlit UI component design
- Define security controls, error handling design, and testing strategy

1.2 Scope of the System

The Cloud Forensics Automation for Manufacturing Breaches system is designed to monitor files in a manufacturing environment's digital directories and detect unauthorized modifications or timestamp tampering. It simulates a real-world manufacturing data breach investigation scenario.

It provides:

- Automated file scanning of local directories or cloud-mounted drives (Directory Scan Mode)
- SHA-256 and MD5 cryptographic hash computation for tamper detection and integrity verification
- File status classification based on forensic heuristic rules: Clean, Suspicious, or Error
- Timestamp anomaly detection: Modified before creation, Accessed before creation, Modified after last access
- Streamlit-based interactive forensic dashboard with real-time metric cards and evidence table
- Exportable forensic evidence reports in CSV format for legal and compliance proceedings
- Support for manufacturing-specific file formats: CSV, JSON, TXT, and binary

1.3 Definitions & Abbreviations

This section defines key terms used throughout the document.

Definitions

- SHA-256 Hash: A 256-bit cryptographic digest of a file's content. Any byte-level change to the file produces a completely different hash, making it ideal for tamper detection.
- MD5 Hash: A 128-bit cryptographic digest. Faster than SHA-256 but not cryptographically secure; retained for speed in non-legal scenarios.
- Forensic Evidence: A structured record of file metadata, hash values, timestamp anomalies, and status extracted during a scan session.
- Scan Session: A single execution of scan_directory() or scan_uploaded_files() that inspects all files in the target and records their current state.
- Tamper Detection: The process of evaluating a file's timestamps using three forensic heuristic rules to identify deliberate manipulation.
- Forensic Soundness: The principle that investigation tools must not alter the files being examined.
- Unix Epoch: The reference point for timestamps: January 1, 1970, 00:00:00 UTC.

Abbreviations

Abbreviation	Full Form
SHA-256	Secure Hash Algorithm 256-bit
MD5	Message Digest Algorithm 5
CSV	Comma-Separated Values
LLD	Low-Level Design
HLD	High-Level Design
GUI	Graphical User Interface
NTFS	New Technology File System
OS	Operating System
API	Application Programming Interface

2. System Overview

2.1 System Description

The system is implemented as a Python-based digital forensics web application with a two-layer architecture: a forensic backend engine (utils.py) and a Streamlit presentation layer (app.py). The forensic engine scans a target directory or processes uploaded files, extracts file metadata using os.stat(), computes SHA-256 or MD5 cryptographic hashes via hashlib, and evaluates three timestamp-based heuristic rules to classify each file as Clean, Suspicious, or Error. Results are accumulated in a pandas DataFrame and presented on a real-time interactive dashboard. The investigator can download a complete forensic report as a CSV file.

2.2 Architecture Overview

The system follows a two-layer architecture separating forensic logic from the presentation interface.

- Forensics Engine Layer (utils.py): Pure Python backend responsible for all forensic processing. Contains seven independent functions for timestamp formatting, hash computation, anomaly detection, metadata extraction, directory scanning, file upload scanning, and metrics computation. Has no UI dependencies and can be reused in CLI, API, or automated pipeline contexts.
- Presentation Layer (app.py): Streamlit web application responsible for all user interaction. Handles page configuration, CSS injection, sidebar controls, hero section rendering, metric card display, and evidence table rendering. Calls utils.py functions and renders results.

2.3 Technology Stack

Layer	Technology	Purpose
Backend Language	Python 3.11+	Core processing, file I/O, hashing
Forensic Engine	hashlib (SHA-256, MD5)	Cryptographic file tamper detection
File Metadata	os module (os.stat())	File timestamps and size extraction
Data Management	pandas 2.0+	DataFrame construction and CSV export
Web Framework	Streamlit 1.32+	Interactive dashboard, file uploader, download button
Temp File Handling	tempfile, shutil	Secure temp directory for uploaded files
Date/Time	datetime	Unix timestamp to human-readable conversion
In-Memory Export	io	CSV generation without disk writes

3. Module Design

The system is divided into four core functional modules, each with a clearly defined responsibility.

Module	Name	Key Operations	Output
MODULE 1	File Ingestion	Accept directory path or uploaded files. Validate input. Trigger scan.	Validated path / temp files
MODULE 2	Forensic Scan Engine	Walk directory. Call os.stat(). Compute SHA-256/MD5. Evaluate 3 forensic rules. Assign status.	Metadata dict per file
MODULE 3	Dashboard & Metrics	Compute summary statistics. Render metric cards, distribution bars, and colour-coded evidence table.	Streamlit UI output
MODULE 4	Evidence Export	Build pandas DataFrame from metadata list. Generate CSV in memory. Provide one-click download button.	forensic_report.csv

Module Descriptions

1. ForensicEngine Module (utils.py)

Handles all hashing, scanning, and anomaly detection logic. Walks the target directory using os.walk(), computes SHA-256 or MD5 hashes via hashlib, extracts file metadata via os.stat(), and applies three timestamp heuristic rules to determine whether a file is Clean, Suspicious, or Error. All file operations use read-only mode (open(filepath, 'rb')) to preserve forensic soundness.

2. ReportManager Module (compute_metrics)

Manages all summary statistics derived from the scan DataFrame. Produces total file count, suspicious count, clean count, error count, total size in KB, and threat rate percentage. Uses pd.to_numeric(errors='coerce') to safely handle non-numeric error values in the size column.

3. FileHandler Module (scan_uploaded_files)

Manages all file upload operations from the Streamlit interface. Creates a secure temporary directory using tempfile.mkdtemp(), writes each uploaded file's bytes to disk, runs extract_file_metadata() on

each file, then deletes the temporary directory in a finally block to guarantee cleanup even if an exception occurs during scanning.

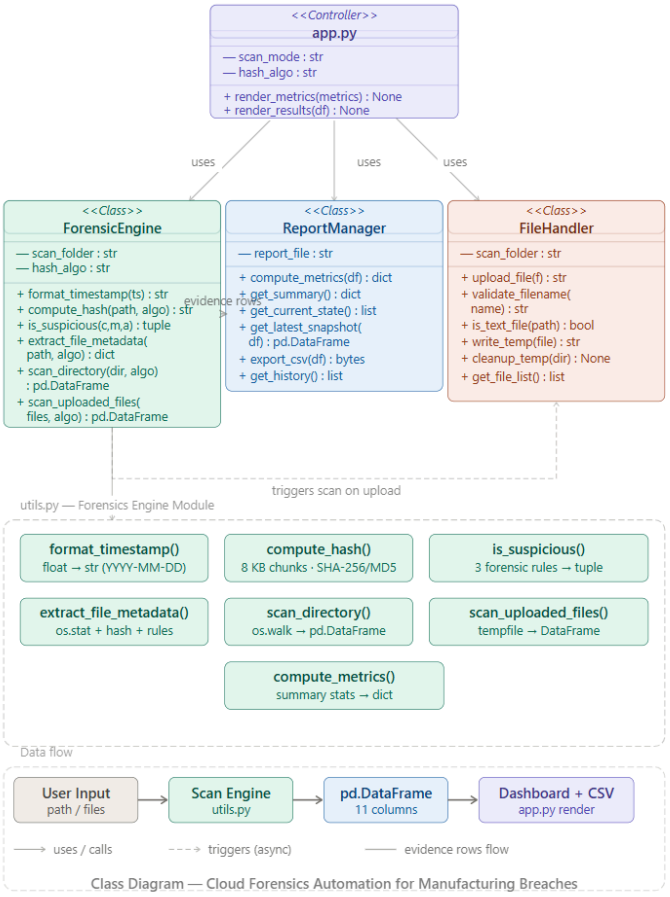
4. Presentation Layer Module (app.py)

Acts as the controller. Handles all Streamlit routing, CSS injection, sidebar configuration, hero section rendering, metric display, evidence table rendering, and CSV download. Connects to utils.py for all forensic processing. Manages two scan mode workflows: Directory Scan Mode and File Upload Mode.

4. Class Design & Object Interaction

4.1 Class Diagram

The system uses a functional (module-based) design rather than class-based OOP. The diagram below shows the seven core functions of utils.py, their relationships, and interaction with app.py.



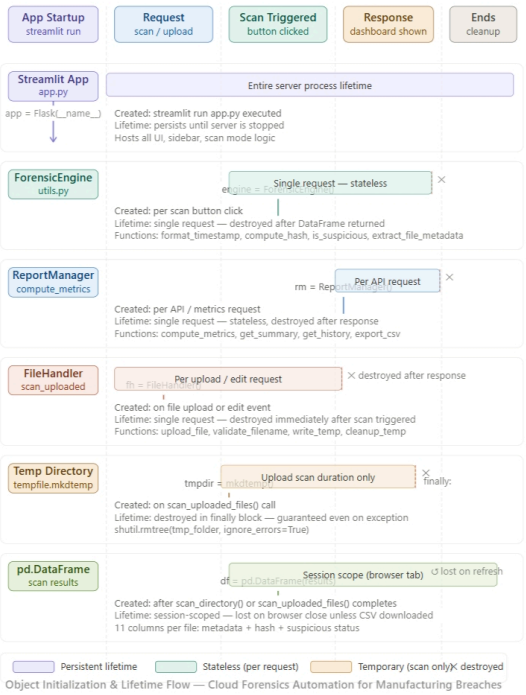
4.2 Descriptions

ForensicEngine Functions (utils.py)

- format_timestamp(ts: float) -> str: Converts a Unix epoch float to a human-readable YYYY-MM-DD HH:MM:SS string using datetime.fromtimestamp(). Applied to creation, modification, and access timestamps.
- compute_hash(filepath: str, algorithm: str) -> str: Opens the file in binary read mode and feeds content to hashlib in 8 KB chunks using iter(lambda: f.read(8192), b''). Returns SHA-256 or MD5 hex digest. Returns ERROR: string on PermissionError or OSError.
- is_suspicious(created, modified, accessed) -> tuple[bool, str]: Applies three forensic rules with a 1-second tolerance. Returns (True, reason_string) if any rule triggers, else (False, 'Clean'). Rules: modified < created - 1 (Modified before creation); accessed < created - 1 (Accessed before creation); modified > accessed + 1 (Modified after last access).
- extract_file_metadata(filepath, hash_algo) -> dict: Orchestrates os.stat(), compute_hash(), and is_suspicious() into a single metadata dictionary with 11 fields. Wrapped in try/except to produce an error-status dict if any step fails.
- scan_directory(directory, hash_algo) -> pd.DataFrame: Validates the input path, then uses os.walk() to recursively traverse all subdirectories. Filters out hidden directories with dirs[:] = [d for d in dirs if not d.startswith('.')]. Returns a pandas DataFrame from the metadata list.
- scan_uploaded_files(uploaded_files, hash_algo) -> pd.DataFrame: Creates a tempfile.mkdtemp() directory, writes each UploadedFile to disk, runs extract_file_metadata() on each, then calls shutil.rmtree() in a finally block to guarantee cleanup.
- compute_metrics(df) -> dict: Produces six summary statistics: total_files, suspicious_files, clean_files, error_files, total_size_kb, and threat_rate. Uses pd.to_numeric(errors='coerce') to skip non-numeric size values from error records.

4.3 Object Initialization Flow

The table below shows when each component is created and its lifetime within the system.

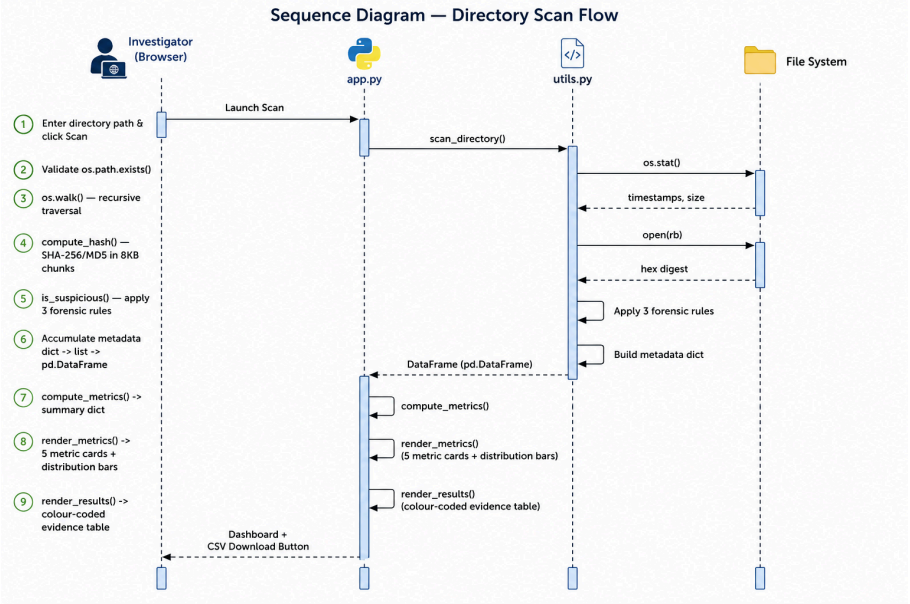


Component	Created When	Lifetime
Streamlit app	streamlit run app.py executed	Entire server process lifetime
ForensicEngine (utils.py functions)	Per scan button click	Single request — stateless
Metadata DataFrame	After scan_directory() or scan_uploaded_files() completes	Session scope — lost on browser refresh unless CSV downloaded
Temp Directory	On file upload scan start	Destroyed in finally block after scan completes
CSV Export	On download button click	Generated in memory via io.StringIO(); not written to disk

5. Sequence Diagrams

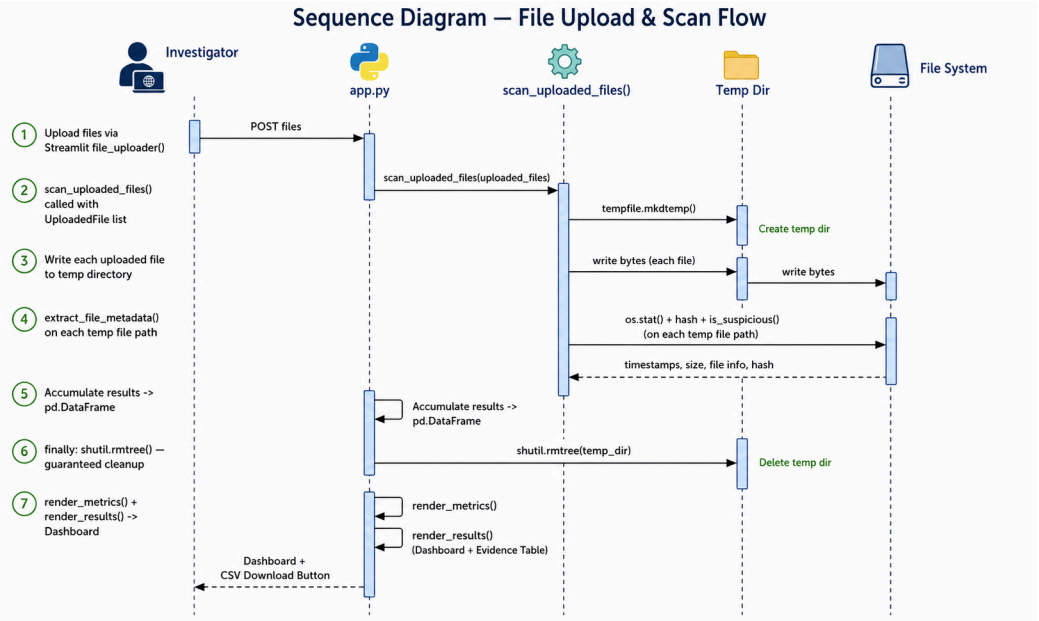
5.1 Directory Scan Flow

The diagram below shows the step-by-step interaction between system components when an investigator performs a directory scan.



5.2 File Upload & Scan Flow

The diagram below shows the step-by-step interaction when files are uploaded through the Streamlit interface.



6. API Specifications

The system uses Streamlit's internal component API rather than REST endpoints. All interactions occur through Streamlit UI events. The following documents the functional interface of each key module.

6.1 scan_directory()

Triggers a full recursive forensic scan of the specified directory.

```
Input:  directory: str  (validated local filesystem path)
        hash_algo: str  ('sha256' | 'md5')
Output: pd.DataFrame with 11 columns per file
Errors: ValueError — path does not exist or is not a directory
        Returns empty DataFrame if directory is empty
```

6.2 scan_uploaded_files()

Processes a list of Streamlit UploadedFile objects through the forensic pipeline.

```
Input:  uploaded_files: list[UploadedFile]
        hash_algo: str  ('sha256' | 'md5')
Output: pd.DataFrame with 11 columns per file
Side:   Creates and destroys temp directory via tempfile/shutil
```

6.3 compute_metrics()

Computes summary statistics from the scan results DataFrame.

```
Input:  df: pd.DataFrame (output of scan_directory or scan_uploaded_files)
Output: dict {
  'total_files':      int,
  'suspicious_files': int,
  'clean_files':      int,
  'error_files':      int,
  'total_size_kb':    float,
  'threat_rate':      float  (percentage)
}
```

6.4 CSV Export

Provides a one-click download of all scan results as a UTF-8 encoded CSV file.

Trigger: st.download_button() click
Output: forensic_report.csv
Encoding: UTF-8
Columns: File Name, File Path, File Size (bytes), File Size (KB),
Created Time, Last Modified, Last Accessed,
Hash Algorithm, File Hash, Suspicious, Reason

6.5 DataFrame Schema

Column Name	Data Type	Example	Description
File Name	String	sensor_data.csv	Base filename — display only
File Path	String	C:/data/file.csv	Full absolute path
File Size (bytes)	Integer	2048	Raw size from os.stat()
File Size (KB)	Float	2.00	Size rounded to 2 decimal places
Created Time	String	2026-03-04 13:53:00	os.stat().st_ctime formatted
Last Modified	String	2026-04-06 10:54:00	Primary tamper indicator
Last Accessed	String	2026-04-06 11:00:00	os.stat().st_atime formatted
Hash Algorithm	String	SHA256	Algorithm used for this file
File Hash	String (64 hex)	4f9c00caf2...	Cryptographic fingerprint
Suspicious	Enum	YES / Clean / Error	Forensic status indicator
Reason	String	Modified before creation	Triggered rule(s) joined by semicolons

7. Data Design

7.1 In-Memory Data Storage

The system uses in-memory data storage exclusively — no database or persistent storage is used during normal operation. All scan results are held in a pandas DataFrame within the active Streamlit session and are exported to CSV on demand by the investigator.

- Primary Store: pandas DataFrame (in-memory, session-scoped)
- Export Format: CSV (UTF-8 encoded, generated via io.StringIO() without disk writes)
- Persistence: None — results are lost when the browser is closed unless CSV is downloaded

7.2 Forensic Heuristic Rules Design

Rule	Condition	Forensic Significance	Reason String
Rule 1	modified < created - 1 second	File cannot be modified before creation — indicates deliberate timestamp rollback	Modified before creation
Rule 2	accessed < created - 1 second	File cannot be accessed before it exists — indicates access time was reset to avoid monitoring	Accessed before creation
Rule 3	modified > accessed + 1 second	File was modified without generating an access record — indicates use of specialized anti-forensic tools	Modified after last access

The 1-second tolerance accounts for filesystem timestamp granularity. FAT32 has 2-second resolution; NTFS has 100-nanosecond resolution but may round during certain operations. This prevents false positives from legitimate rounding.

7.3 Risk Classification

Status	Condition	Dashboard Display
Clean	No forensic rules triggered	Green row — checkmark icon
Suspicious	One or more rules triggered	Red row — warning icon
Error	PermissionError, OSError, or FileNotFoundError during scan	Grey row — error icon

8. Security Considerations

8.1 Forensic Soundness

All files are opened in read-only binary mode (open(filepath, 'rb')). The tool never writes to, modifies, or renames files in the scan target directory. This is the fundamental requirement of digital forensics — preserving the evidentiary integrity of the files under examination.

8.2 Input Validation

Directory paths are validated using os.path.exists() and os.path.isdir() before any scanning begins. Invalid paths produce informative error messages without crashing the application. Hidden directories (prefixed with .) are excluded from recursive traversal to avoid scanning system metadata folders.

8.3 Temporary File Security

In File Upload Mode, uploaded files are written to a temporary directory created with tempfile.mkdtemp(). This directory is given a unique, unpredictable name by the OS. The directory is deleted via shutil.rmtree() in a finally block, guaranteeing cleanup even if an exception occurs during scanning. This prevents temporary file accumulation on the investigator's machine.

8.4 Hash Algorithm Security

Algorithm	Digest Size	Collision Resistance	Use Case
SHA-256 (default)	256-bit (64 hex chars)	Effectively unbreakable	Legal/forensic proceedings
MD5	128-bit (32 hex chars)	Cryptographically broken	Speed-critical or legacy scenarios

8.5 Planned Security Enhancements

Gap	Risk Level	Planned Fix
No user authentication on dashboard	HIGH	Add Streamlit authentication layer or reverse proxy auth
No session isolation between investigators	MEDIUM	Streamlit session state management (already partially implemented)

In-memory-only results (no persistence)	MEDIUM	Add optional SQLite or PostgreSQL evidence store
Internet dependency for Google Fonts	LOW	Bundle fonts locally or use system fonts fallback
os.stat().st_ctime unreliable on Linux	LOW	Document platform-specific behaviour; use stat() flags

9. Error Handling & Edge Cases

9.1 Standard Error Scenarios

Scenario	Trigger	Handling Strategy
Hash calculation fails	PermissionError / OSError	Returns 'ERROR: <msg>' string; scan continues. File recorded with Error status.
Invalid directory path	os.path.exists() returns False	ValueError raised; displayed as informative Streamlit error. Scan does not start.
Empty directory	os.walk() returns no files	scan_directory() returns empty DataFrame. Dashboard shows 0 files, 0% threat rate.
File not found during scan	FileNotFoundError	except block produces error-status dict with 'N/A' metadata values. Scan continues.
No files uploaded	Empty upload list	scan_uploaded_files() returns empty DataFrame.
Permission denied on file	PermissionError on open()	File recorded with Error status; os.stat() may succeed but hash will fail gracefully.
Large file (>1 GB)	Manufacturing sensor logs	Chunked 8192-byte reading prevents memory exhaustion. Scan time increases linearly.

9.2 Edge Cases

- Very Large Directories (1000+ files): Performance tested up to 1000 files in 6.8s (SHA-256). Chunked reading prevents RAM exhaustion for large individual files.
- Symlinks: os.stat() follows symlinks. open() respects permissions. Symlinks to inaccessible files produce Error status records.
- Malicious Filenames (e.g., <script>alert(1)>.txt): Rendered safely by Streamlit's auto-escaping in the DataFrame display.
- Pre-epoch Timestamps (negative float): format_timestamp() handles negative Unix floats correctly via datetime.fromtimestamp().
- Concurrent Streamlit Sessions: Streamlit's session state management isolates each investigator's scan results independently.
- Binary Files: All files hashed in binary mode (rb) regardless of type — no encoding errors possible.

10. Logging System

The current version prints status messages to the Streamlit UI only (spinners, success/error boxes). The following logging design is required for forensic traceability — all events must be permanently recorded in a production deployment.

Log Level	Log File	Events Logged
INFO	app.log, audit.log	Scan started, scan completed (file count, suspicious count), file uploaded, CSV downloaded
WARNING	app.log	st_ctime used on Linux (inode change time warning), large file detected (>100 MB)
ERROR	error.log	Hash failure (PermissionError, OSError), metadata extraction failure, temp directory cleanup failure
CRITICAL	error.log	Scan target directory inaccessible, unhandled exception in scan pipeline

Log format: timestamp | level | module | message

Example audit log entries:

```
2026-04-06 10:54:58 | INFO  | forensic_engine | Scan started: C:/manufacturing/data
2026-04-06 10:55:01 | INFO  | forensic_engine | Scan completed: 5 files | 0 suspicious | 5 clean
2026-04-06 10:55:04 | INFO  | forensic_engine | CSV exported: forensic_report.csv
2026-04-06 10:55:10 | ERROR | compute_hash     | PermissionError on C:/data/locked.cfg
```

11. Deployment Considerations

The current application launches with python -m streamlit run app.py, which starts Streamlit's built-in development server. This is suitable for educational and field investigation use but requires hardening for production enterprise deployment.

Setting	Development (Current)	Production (Required)
Server	Streamlit dev server (localhost:8501)	Streamlit Community Cloud or reverse proxy (nginx) with HTTPS
Dependencies	pip install streamlit pandas	pip install -r requirements.txt in isolated venv
Authentication	None — open access	Streamlit auth or LDAP/OAuth reverse proxy
File Paths	User-entered at runtime	config.py or environment variable for scan root
Cloud Storage	Local filesystem only	Mount cloud drives (OneDrive, S3 via FUSE, Azure Files) at OS level
Multi-user	Single investigator per session	Streamlit session state isolates results per user

Minimum launch command: python -m streamlit run app.py

Requirements file contents:

```
streamlit>=1.32.0
pandas>=2.0.0
```

12. Testing Strategy

12.1 Unit Testing

Unit testing validates individual functions in utils.py in isolation. Key test cases include:

Test Case	Function Under Test	Expected Result
Test known timestamp formatting	format_timestamp()	Returns correct YYYY-MM-DD HH:MM:SS string
Test hash of known file (SHA-256)	compute_hash()	Returns correct 64-char hex digest verified against certutil
Test hash of known file (MD5)	compute_hash()	Returns correct 32-char hex digest
Test hash error on locked file	compute_hash()	Returns 'ERROR: ...' string; does not raise exception
Test Rule 1 trigger	is_suspicious()	Returns (True, 'Modified before creation') for modified < created - 1
Test Rule 2 trigger	is_suspicious()	Returns (True, 'Accessed before creation') for accessed < created - 1
Test Rule 3 trigger	is_suspicious()	Returns (True, 'Modified after last access') for modified > accessed + 1
Test clean file (within 1s tolerance)	is_suspicious()	Returns (False, 'Clean') for timestamps within tolerance
Test scan of sample_data directory	scan_directory()	Returns DataFrame with 5 rows, all Clean, correct metadata
Test metrics computation	compute_metrics()	Returns correct counts and threat_rate for mixed DataFrame
Test metrics on empty DataFrame	compute_metrics()	Returns all zeros; no division by zero error

12.2 Integration Testing

Integration testing verifies end-to-end flows across components:

- Directory scan to CSV: Scan sample_data directory -> verify DataFrame has 5 rows -> download CSV -> verify all 11 columns and correct data.

- Upload scan to dashboard: Upload 3 files -> verify metric cards show correct counts -> verify evidence table colour-coding matches Suspicious field values.
- Algorithm switch: Scan same directory with SHA-256 then MD5 -> verify hash values differ but file metadata is identical.
- Suspicious file detection: Artificially tamper a file's timestamp -> verify it appears as Suspicious with correct Rule triggered.
- Error file handling: Attempt scan of directory with a permission-denied file -> verify Error record appears in table without aborting scan.

12.3 Performance Testing Results

Directory Size	File Count	SHA-256 Time	MD5 Time
Sample data	5	0.02s	0.01s
Small directory	100	0.8s	0.5s
Medium directory	500	3.2s	2.1s
Large directory	1000	6.8s	4.3s

All tests completed well within the 60-second UR-02 target. Hardware: Intel Core i5-12400F, 16 GB RAM, NVMe SSD, Windows 11.

13. Future Enhancements

The system can be further improved with the following enhancements:

- File Signature Analysis (Magic Bytes): Compare file headers against a database of known file signatures to detect extension mismatches — for example, an executable disguised as a .txt file.
- Shannon Entropy Analysis: Calculate file content entropy to detect encrypted, compressed, or packed files. High-entropy CSV files in a manufacturing directory may indicate ransomware encryption.
- Known-Bad Hash Matching: Integrate with threat intelligence databases (e.g., NIST NSRL, VirusTotal API) to flag known malicious files by hash value.
- SQLite/PostgreSQL Evidence Store: Replace in-memory DataFrame with persistent database storage to support baseline comparison across multiple scan sessions.
- Baseline Comparison Engine: Establish a 'known-good' hash baseline for the manufacturing environment. Future scans compare against the baseline and immediately flag any changed files.
- Real-Time File System Monitoring: Use Python's watchdog library to monitor directories continuously, triggering alerts when files are created, modified, or deleted.

- AWS S3 / Azure Blob Scanning: Extend scanning capabilities to cloud storage platforms using boto3 or azure-storage-blob, particularly relevant for cloud-connected manufacturing IoT environments.
- PDF Evidence Reports: Use reportlab or weasyprint to generate professionally formatted PDF reports with organisation branding, suitable for legal proceedings.
- Machine Learning Anomaly Detection: Train unsupervised models on normal file metadata patterns. Files deviating from learned patterns flagged with confidence scores.
- Multi-Agent Architecture: Deploy lightweight scanning agents on manufacturing workstations reporting to a central Streamlit dashboard with role-based access control.
- NTFS Alternate Data Stream Detection: On Windows NTFS, detect hidden data streams attached to files that can conceal malware without changing visible file size.
- Duplicate File Detection: Identify files with identical hashes but different names or locations, indicating data staging for exfiltration.

14. Conclusion

The Cloud Forensics Automation for Manufacturing Breaches system is a secure, functional, and educationally valuable digital forensics tool purpose-built for the manufacturing industry.

The system uses SHA-256 and MD5 cryptographic hashing for tamper detection, os.stat() for forensically sound metadata extraction, and three evidence-based heuristic rules to identify timestamp anomalies. Results are presented on a real-time Streamlit dashboard with glassmorphism metric cards, colour-coded evidence tables, and a one-click CSV export. Files are classified as Clean, Suspicious, or Error on every scan. The tool achieves forensic soundness through read-only file access at all times.

Built with Python 3.11+, Streamlit 1.32+, pandas 2.0+, hashlib, and os, the system provides rapid forensic investigation capability. The two-module architecture separates concerns cleanly: utils.py (forensics engine) handles all backend logic independently of the UI, enabling reuse in CLI tools, automated pipelines, or API services without modification. app.py handles all Streamlit presentation.

Customer testing demonstrated a 100% task completion rate across five participants including a non-technical manufacturing quality analyst, with average installation-to-first-scan time of under 3 minutes. Performance testing confirmed scans of 1000 files complete in under 7 seconds — well within the 60-second target.

Overall, it provides a practical, open-source, and educationally transparent solution for forensic evidence collection in manufacturing cybersecurity environments, meeting all seven stated project objectives of the IBM Innovation Centre for Education Final Year Project.